WMO Guide to Free and Open Source Software

World Meteorological Organization

Date: 2026-03-23

Version: 0.1.0

Document status: DRAFT

Document location: https://wmo-im.github.io/wmo-foss-guide/guide/wmo-foss-guide-DRAFT.html

WMO publication location: TBD

Standing Committee on Information Management and Technology (SC-IMT)[1]

Commission for Observation, Infrastructure and Information Systems (INFCOM)[2]

Copyright © 2025 World Meteorological Organization (WMO)

Table of Contents

Introduction . . . 3
- Audience . . . 3
- Scope . . . 3
- Background . . . 3
- Data policy considerations (TODO Athina) . . . 4
  - FAIR data . . . 4
  - Open data policies . . . 4
  - Enabling WMO Unified Data Policy via software . . . 4

Guidelines . . . 5
- WMO Members . . . 5
  - Using FOSS . . . 5
  - Contributing to FOSS . . . 5
  - Managing FOSS activities . . . 6
  - Core Benefits of Open Source Software . . . 7
  - Profiling competencies . . . 8
  - Infrastructure considerations . . . 8
- WMO Activities . . . 8
  - Coordination, alignment and support . . . 8
  - Standards compliance . . . 12
  - Software review and evaluation . . . 13
  - Reference Implementations . . . 14
  - Case study: WIS2, FOSS, and agile development . . . 16

References . . . 18

Annex A: FOSS evaluation rubric . . . 19

Annex B: Examples of FOSS managed / offered by Member organizations . . . 20

Introduction

Open source software has become increasingly crucial for WMO and its Members. No longer a "niche" industry, FOSS now powers major initiatives and services worldwide, and is backed by industry support.

FOSS solutions play a pivotal role as accelerators for the implementation of Early Warnings for All, a key WMO initiative aimed at protecting every person on Earth with life-saving early warning systems by 2027. By providing accessible, customizable, and cost-effective tools, FOSS enables Members to rapidly deploy and adapt early warning systems to their specific needs and contexts. This approach is particularly crucial for developing countries (LDCs) and small island developing states (SIDS), where resource constraints often hinder the implementation of proprietary solutions.

Moreover, FOSS initiatives serve as powerful catalysts in supporting Members' efforts towards digital transformation. As National Meteorological and Hydrological Services (NMHSs) worldwide strive to modernize their operations and services, FOSS now offer a flexible and scalable foundation for innovation. They enable Members to leverage cutting-edge technologies, collaborate on development, and share best practices, thus accelerating their digital transformation journeys while optimizing resource utilization.

Audience

The document is intended for WMO Members who are involved in software development activities in their respective organizations (developers, managers/decision makers) as well as WMO Secretariat in the coordination, alignment and support of FOSS within WMO.

Scope

This document provides guidance on FOSS for WMO Members and WMO Secretariat.

Background

FOSS development and use among WMO Members has been a longstanding activity. Typical FOSS implementations included decoders and encoder libraries of WMO BUFR and GRIB formats (e.g. ecCodes, degrib, libecbufr, etc.), as well as data dissemination services (THREDDS, GDAL, ERDDAP, etc.).

In addition, as part of the global open data/open science movement, numerous government organizations have put forth using FOSS by default; this includes use of exiting FOSS tools as well as developing of same, working in the open by default. Free software development platforms and tools (for example, GitHub) have greatly lowered the barrier for sharing software for research, development and operations for all.

As part of the development of the WMO Information System (WIS2), FOSS implementation has been a significant activity, as exemplified by a number of Reference Implementations of WIS2 Standards (such as wis2box, wis2-gdc, etc.). The development of software in the open during the pre-operational phases of WIS2 proved valuable in testing WIS2 Standards put forth in Technical

Regulations while in development, promoting the “release early, release often” philosophy of agile and iterative development. While FOSS is resident in numerous WMO Member organizations, guidance and coordination becomes more vital to ensure Member services are provided with security, privacy and safety in mind for WMO Members and beyond.

Data policy considerations (TODO Athina)

While talking about Open Source Software, open data plays an equally important role. The following section addresses the crucial complementary role of open data to Open Source Software and how Open Source Software can be an enabler for the WMO Unified Data Policy [LINK - https://wmo.int/wmo-unified-data-policy-resolution-res1].

FAIR data

Making data FAIR – Findable, Accessible, Interoperable and Reusable – is an important aspect in today’s digitally connected world. This is especially true and important for the meteorological domain, where local, regional and global models rely on data from different source, organisations and regions. Data that lives in silos and cannot be accessed has a very limited use.

Open data policies

- Important for advancing trust in data
- Important for traceability of results etc
- Examples:
  - https://digital-strategy.ec.europa.eu/en/policies/legislation-open-data
  - https://opendatapolicylab.org/
  - https://repository.opendatapolicylab.org/
  - https://www.ecmwf.int/en/forecasts/datasets/open-data
  - https://www.ecmwf.int/en/forecasts/access-forecasts/access-real-time-open-data
  - https://data.ecmwf.int/

Enabling WMO Unified Data Policy via software

https://wmo.int/wmo-unified-data-policy-resolution-res1

[1] https://community.wmo.int/governance/commission-membership/commission-observation-infrastructures-and-information-systems-infcom/commission-infrastructure-officers/infcom-management-group/standing-committee-information-management-and-technology-sc-imt

[2] https://community.wmo.int/governance/commission-membership/infcom

Guidelines

WMO Members

Using FOSS

Risks

- **License compliance risks** Open source licenses often have widely varying requirements regarding commercial use, redistribution, and code modification. Certain licenses include restrictive clauses, such as mandating that derivative works be made open source or that original copyright notices be preserved in full. Violations may lead to legal disputes and intellectual property issues.
- **Security and continuity risks**: Open source software from niche projects or less-established projects may pose security risks due to limited formal code audits. Additionally, the complexity of open source software supply chains creates vulnerabilities that could be exploited to implant malicious code. These risks are amplified in meteorological applications, especially when handling sensitive observational data or real-time forecasting models.
- **Maintenance risks**: Fixes for vulnerabilities rely on community responsiveness. If an open source project is discontinued, known vulnerabilities may remain unaddressed.
- **Support risks**: Most open source software lacks official support. As a result, timely response and effective resolutions of critical operational system failures cannot be guaranteed, potentially leading to interruptions in meteorological operation and data loss.

Hidden costs

- **Technical adaptation and integration costs**: Human resources must be allocated for secondary development (customization), system interface integration, and compatibility testing to ensure the software aligns with existing business systems.
- **Long-term maintenance and version management costs**: Dedicated personnel are needed to track version updates, conduct testing, and manage migrations. If a project is discontinued, taking over its maintenance internally entails high technical challenges and added operational complexity.
- **Talent acquisition and training costs**: Recruiting skilled professionals can be expensive, and ongoing in-house training is required to maintain the necessary expertise.
- **Compliance audit and management costs**: It is necessary to maintain an inventory of all open source components and perform regular compliance audits. Large meteorological organizations may also need to develop dedicated management platforms, with costs increasing as the number of components grows.

Contributing to FOSS

National policies

The adoption and development of FOSS within NMHSs can be significantly strengthened through

supportive national-level policies. Several WMO Members have already introduced digital government strategies that encourage the use of open standards, open data and open-source software as default options within public administrations. Where such frameworks exist, NMHSs can leverage them to increase efficiency, reduce vendor lock-in and ensure long-term sustainability of operational systems.

National FOSS-related policies may include:

- "Open by default" procurement rules requiring public institutions to consider FOSS alternatives alongside proprietary solutions when acquiring software. In some countries, like Romania, when you purchase "on demand" software development services, the source code should be transferred to the customer along with all the IPR associated with the software. Making the source code available under an open-source license is highly recommended.
- Guidelines for software development funded from public budget, encouraging publication of code under approved open-source licenses.
- Policies promoting interoperability and open standards, which naturally align with FOSS implementations within WIS2 and other WMO technical frameworks.
- Capacity-building programs that enable staff to acquire skills for maintaining and developing open-source tools, reducing long-term operational dependency on external vendors.
- Security and lifecycle requirements to ensure that open-source components used in critical systems follow maintenance, patching, auditing and version management rules.

Where no national regulations exist, NMHSs can still develop institution-level policies or internal guidelines to support responsible FOSS adoption:

- Establish internal rules for licensing, contribution approval and repository management.
- Define procedures for evaluating FOSS components (governance, community health, maintenance model).
- Incorporate sustainability planning (including contribution back to upstream projects when fixes or new features are developed internally).
- Promote transparency by adopting open repositories for research software and operational tools.
- Events/hackathons (e.g. OGC/OSGeo/ASF Joint Sprints) TODO Vasile
    - By product: connection/collab
- Regulations / risk / constraints / considerations
    - Features, maintenance, project

Managing FOSS activities

In some cases, Members developing their own FOSS in support of WMO obligations and activities may wish to make the software freely available to the public. The current FOSS landscape and infrastructure provides a strong support mechanism for doing FOSS development in the open, using services such as GitHub, GitLab, Codeberg and numerous other platforms.

In the same spirit as contributing to FOSS, where no national regulations exist, NMHSs can still

develop institution-level policies or guidelines in support of making FOSS available to the public. The following are (some) key aspects to consider:

- Establish an "architecture of participation":
  - Who will be responsible for development, release management, and accepting external contributions?
  - How will commit access be governed and managed? Does the project need a project steering committee or group of stakeholders to help drive the project?
  - How will external collaboration be addressed? For example, will the project have a bug or issue tracker which can be used to identify new feature development, defects and/or bugs. Will there be a dedicated discussion forum or mailing list for wider discussion?
  - Make clear how collaboration will work to the public (for example, by providing a `CONTRIBUTING.md` document for GitHub)
  - Make clear how security issues should be addressed (mitigation/release policies/timelines, contact points, etc.)
  - What is the release schedule for new versions?
- Choose a license that the software will be made available with (such as the Apache License, Version 2.0, MIT, etc.). It is strongly advised to consider and consult organization policies to ensure proper licensing and understanding risk, responsibility and liability
- Choose the mechanism by which the software will be made available (GitHub, GtiLab, Codeberg, organizational website, etc.)
- Ensure that a `README` is available with the code stating a clear mission and purpose
- Identify any supported WMO Technical Regulations
- Ensure that the project has proper documentation for users (downloading, installing, configuring, running), as well as additinal documentation for developer or othe audience types (administrators, etc.)
- Ensure the code is versioned (for example using Git as a version control system)

Core Benefits of Open Source Software

The core benefits of open source software can be summarized on three dimensions: autonomy and control, cost optimization, and ecosystem flexibility—offering both practical advantages and adaptability. Details are as follows:

- **Exceptional autonomy and control**: The code is open and licenses are flexible, allowing meteorological organizations to perform deep customization as needed, directly modifying the code to add or remove functional modules to suit unique operational scenarios (e.g., regional weather forecasting adjustments). This level of access also allows them to independently identify and fix vulnerabilities, optimize performance for massive meteorological data processing, and control the technical roadmap of core systems. Certain permissive licenses (e.g., MIT License, Apache License 2.0) even permit using modified code as closed source in commercial products, balancing customization needs with operational requirements.
- **Significant cost advantages**: No software license fees or subscription fees are required,

suitable for national meteorological services as well as deployments in regional forecasting centers. The availability of source code and community documentation reduces trial-and-error costs during secondary development, making necessary human resource investment more manageable. Meteorological organizations can also choose free community support or affordable third-party services (specialized in meteorological applications) instead of expensive vendor support packages.

- **Avoiding vendor lock-in risks**: Free from proprietary interfaces and data format dependencies, enabling free choice of deployment environments (e.g., private or public clouds). Even if the original project is discontinued, maintenance can be taken over independently or by a third party. Moreover, multiple service providers are often support the same open source software, strengthening the user's bargaining power.
- **Excellent cross-platform portability**: Most open source tools run on multiple operating systems (e.g., Windows and Linux), eliminating the need for separate purchases. Adherence to common technical standards (e.g., WMO GRIB data formats) lowers system integration difficulty. Modular design facilitates containerized deployment and cross-cluster migration, adapting to architectural upgrades in meteorological data centers.

Profiling competencies

Members wishing to use, participate and/or manage FOSS activities should be familiar with basic FOSS principles, and assess their level of participation accordingly. Understanding the architecture of participation of FOSS varies among projects. In addition to domain and software development competencies, staff require the ability to analyze a given FOSS project and/or community, understanding how development, contributions and social and political infrastructure operate. Staff should also be supported to participate in FOSS as part of their ongoing learning and development. This can include attending conferences and/or code sprints and hackathons. The degree to which these skills are developed depend on the level of FOSS investment by a given Member organization.

Infrastructure considerations

(TODO TBD)

WMO Activities

Coordination, alignment and support

As FOSS has become increasingly embedded in WMO programmes, projects and Member operations, the need for structured coordination at the organizational level has grown accordingly. Historically, software developed, or selected, for use in different projects and programmes has largely been managed on an ad-hoc basis. Independent experts, contractors, and implementing authorities / agencies have typically made independent choices without a common framework for the evaluation, governance, and life-cycle management of FOSS.

Whilst enabling rapid development and innovation, this approach has led to fragmentation and duplication of effort, and in some cases, reliance on unmaintained or insecure software that puts Members services at risk.

The WMO Secretariat, as the body supporting the work of its Members and constituent bodies, has a central role in coordinating FOSS activities across WMO programmes and projects and in supporting Members in the adoption of FOSS. This coordination function should ensure that FOSS initiatives adopted through WMO projects and programmes are aligned with the WMO's regulatory framework and that software used in support of WMO obligations meets appropriate standards of quality and security. Equally important, this coordination should ensure that Members, particularly those with more limited technical capacities, are not left behind.

Governance and organizational functions

Effective FOSS coordination with the WMO requires dedicated governance functions. The following functions are identified as essential:

- **Software evaluation and review**: Providing a consistent process for assessing FOSS used in WMO programmes and projects, applying criteria such as those described in the FOSS evaluation rubric (see FOSS evaluation rubric). This includes evaluating license compatibility, security posture, project and community health, openness of governance, and technical fitness for purpose.
- **Lifecycle oversight**: Tracking the status of FOSS components used across WMO programmes, from initial adoption through operational use to end-of-life, ensuring that migration plans are in place when software is no longer maintained.
- **Standards alignment**: Ensuring that FOSS developed or endorsed within WMO implements relevant Technical Regulations and open standards correctly and consistently.
- **Guidance and advisory support**: Providing practical guidance to Expert Teams, Task Teams, Communities of Practice, and other implementors on FOSS best practices, including licensing, contribution workflows, governance models and release management.

These functions may be carried out by existing WMO bodies, such as the Standing Committees, Expert Teams, and Task Teams. However, these bodies often have a narrow focus on a particular activity or area, are not typically involved in the projects implementing or using FOSS, and often lack the specialist knowledge on FOSS. Alternatively, a dedicated software review function, such as through an Open Source Program Office (OSPO), could be established. Regardless of the organizational form, there is a clear requirement for software governance to be systematic rather than ad-hoc, and that clear, objective criteria exist for evaluating and endorsing FOSS within WMO.

It is important to note that WMO should not seek to develop its own registry of open-source software.

Existing registries and ecosystems (such as those maintained by OSGeo, the Digital Public Goods Alliance, and language-specific package repositories) already serve this purpose. WMO's coordination role should focus on evaluating and recommending software from within these existing ecosystems, rather than duplicating their efforts.

Roles and responsibilities

Coordination of FOSS activities within the WMO involves a range of actors:

- **WMO Secretariat**: Facilitates coordination across programmes and projects, supports the

development and maintenance of FOSS guidance, and manages relationships with external FOSS organizations. The Secretariat also provides continuity and institutional memory for FOSS initiatives that span multiple constituent body sessions.

- **Expert Teams and Task Teams**: Provide domain-specific expertise in evaluating and developing FOSS relevant to their areas of responsibility. Expert Teams and Task Teams may identify candidate software for endorsement or develop Reference Implementations of WMO standards.
- **Communities of Practice**: Serve as collaborative spaces where Members, developers and stakeholders work together on shared FOSS initiatives. Communities of Practice can play an important role in maintaining and evolving software that supports WMO standards, as demonstrated by the development of <<_Reference Implementations within the WIS2 programme and the OpenCDMS project and initiative.
- **Members**: Contribute expertise, code, testing and operational feedback. Members with mature FOSS programmes can serve as mentors and collaborators for those building capacity.

Whilst "Communities of Practice" have been listed above, these do not exist formally within the WMO's framework and constituent bodies, instead operating on an informal basis. Strengthening and formally recognising these communities would have significant benefit.

FOSS project life cycle

FOSS projects and software typically progress through several stages:

- **Concept and prototyping**: An initial requirement is identified and ideas are explored. These may often be part of the development of a new standard, such as the WIS2.0 standards, or in response to an operational need. At this stage, the software may be experimental and maintained by a small number of developers.
- **Development and pilot use**: Following initial prototyping, the software gains maturity and may have additional contributors. The software is tested in pilot environments. Parallel to this, the standards that the software is implementing may also gain maturity and undergo further changes.
- **Operational use**: Software, and standards, are ready for production use in operational environments. There will be established governance processes for the FOSS together with comprehensive documentation and contribution guides. The contributor base will have grown and be sustainable.
- **End-of-life and migration**: When software is no longer actively maintained or has been superseded, clear guidance should exist for migrating to alternative solutions, including data migration. End-of-life planning is particularly important for software that Members depend on for operational services.

The WMO can support this lifecycle by providing evaluation criteria at each stage, recognizing software that meets established thresholds of maturity, and facilitating transitions when software reaches end-of-life. Established frameworks such as the OSGeo incubator and graduation process[1] provide a useful model for structuring such a lifecycle within the WMO.

FOSS sustainability and risk management

Sustainability, and long-term maintenance and support, are amongst the most significant challenges in FOSS adoption. Software that is relied upon by Members, especially for operational services, must be maintained over the long-term, with timely security updates and patches, compatibility updates and responsive support. Key risks to be managed include:

- **Bus / retirement factor**: Over-reliance on a single, or small team, for critical software. When supporting FOSS development and adoption, the WMO should encourage a broad contributor base, shared governance, and pooling of resources for the support and development of critical software.
- **Security vulnerabilities**: Software components must be subject to regular security reviews, with clear disclosure and patching policies. Adequate resources must be in place to ensure any identified vulnerabilities are addressed in a timely fashion.
- **Dependency risk**: Software often depends on a chain of upstream libraries and components. Understanding and monitoring these dependencies is essential for assessing overall project health. This includes changes to upstream licensing conditions.
- **Funding and institutional support**: Whilst FOSS may be free of licensing costs there are still costs associated with the development, maintenance, and operation of FOSS software. These costs are often hidden or neglected but should be taken into account.

The WMO has a role in supporting FOSS initiatives and managing risks by promoting and supporting investment in the development and maintenance of FOSS software and in community management. This support may be realised through sources such as the WMO Voluntary Cooperation Programme (VCP), extra-budgetary resources and projects, and through broader stakeholder and donor engagement.

The WMO's coordination role should include monitoring the health of FOSS that is critical to WMO programmes and alerting Members to emerging risks, such as previously promoted projects that show declining activity, unpatched vulnerabilities or loss of key maintainers.

Alignment with the WMO ecosystem

In the WMO context, FOSS initiatives should be aligned with the broader set of WMO activities in supporting Members. This includes:

- **Technical Regulations and standards**: Co-development of Reference Implementations alongside the development and implementation of new regulations and standards; FOSS developed or endorsed should conform with and / or implement WMO standards. The WMO should also promote alignment and interoperability with open standards, such as those developed by the Open Geospatial Consortium (OGC).
- **Cross-programme and cross-project coordination**: FOSS components are often relevant to multiple WMO programmes. Coordination can help avoid duplication, identify opportunities for shared development and ensure that Members benefit from a coherent set of tools rather than competing or overlapping solutions.

Supporting Members

A key function of WMO coordination is ensuring that all Members can benefit from FOSS, regardless of their technical capacity, and making sure that no Member is left behind. This includes:

- **Capacity building**: Providing training, documentation and technical support to help Members evaluate, deploy and maintain FOSS solutions. This is particularly important for developing countries and small island developing states, where resource constraints may limit the ability to engage with FOSS communities directly.
- **Knowledge transfer**: Facilitating the exchange of experience and expertise between Members, for example through workshops, code sprints and hackathons organized alongside WMO events or in collaboration with partner organizations.
- **Visibility and discovery**: Helping Members identify FOSS that is relevant to their needs by curating recommendations and maintaining awareness of available tools within the WMO community, drawing on existing registries and partner ecosystems.

External partnerships

The WMO does not operate in isolation in the FOSS landscape. Established organizations such as the Open Source Geospatial Foundation (OSGeo), the Digital Public Goods Alliance (DPGA), the Apache Software Foundation and others have developed mature frameworks for FOSS governance, incubation and community management. WMO should seek to leverage these existing frameworks rather than developing parallel structures.

Potential areas for collaboration include:

- **Formal partnerships:** Memoranda of Understanding (MoUs) with organizations such as OSGeo can provide a structured basis for collaboration on project governance, incubation and quality assurance.
- **Registration as Digital Public Goods:** Key WMO FOSS implementations may qualify for registration with the DPGA, increasing their visibility and credibility within the broader international development community.
- **Joint events:** Code sprints, hackathons and workshops organized in collaboration with partner organizations (such as OGC/OSGeo/ASF joint sprints or FOSS4G conferences) provide valuable opportunities for capacity building, community engagement and collaborative development.
- **Regional digital innovation hubs**: Regional digital innovation hubs, established in collaboration with Members, national and international development agencies, and regional organizations, can play an important role in the development and implementation of FOSS solutions, with a focus on meeting the needs of Members within a region. These could be co-located, or hosted, by existing regional centres.

Standards compliance

FOSS can also play a significant role as an indicator of the feasibility of Technical Regulation implementation. As part of the standards development phase, developing the associated FOSS implementation(s) aids in determining "fit for purpose" of the specification. In addition, FOSS implementation during standards development allows for an open review and assessment on how

a standard is implemented (supported network protocols, avaiability of tooling/libraries for integration, as well as speed of implementation (if a FOSS implementation during the development of a standard takes a considerable level of effort, why? Is the complexity acceptable for the scope of the standard? Determining these aspects earlier can help improve and adjust the standard while in development (as opposed to formal amendment processes once approved by WMO). The agile and parallel development of WIS2 standards and FOSS is a key example in this context (see Case study: WIS2, FOSS, and agile development).

Another indicator is the number of implementations. For example, the Open Geospatial Consortium (OGC) mandates that a proposed standard requires at least 3 implementations for consideration by its Technical Committee. This provides "proof" to the Technical Committee that the standard can be implemented, and has investment from various projects as early adopters.

In summary, while FOSS is not in scope for being identified as part of a Technical Regulation, it is a strong indicator of the the maturity, applicability and sustainability of a given standard. Implementing FOSS during standards development provides safety in investment, iterative improvements, and confidence for widespread and sustainable adoption and implementation by Members.

Software review and evaluation

The identification and selection of software for WMO-related activities and Member implementations should follow a structured, transparent and repeatable assessment process. This applies equally to FOSS and proprietary solutions and aims to reduce technical, operational and organizational risks while increasing confidence in long-term sustainability, interoperability and compliance with WMO frameworks. Before adopting, recommending or endorsing a software solution, Members and WMO bodies are encouraged to perform a lightweight but systematic project assessment covering at least the following dimensions:

- Purpose and scope alignment. The software should have a clearly defined purpose and scope, aligned with the intended operational, research or regulatory use case. Its role within the overall system architecture (e.g. data production, exchange, discovery, processing or visualization) should be explicit and overlaps or dependencies with existing solutions should be identified.
- Standards and regulatory compliance. The software should demonstrably support relevant WMO Technical Regulations, standards and recommended practices, as applicable (e.g. WIS2, BUFR, GRIB, NetCDF, WMO Core Metadata Profile). Where full compliance is not yet achieved, gaps, limitations and mitigation measures should be clearly documented.
- Licensing and policy compatibility. Licensing terms must be clearly stated and compatible with national regulations, WMO data policies and operational constraints. For FOSS, licenses should be OSI-approved and allow redistribution, modification and operational use without legal ambiguity.
- Project maturity and sustainability. The assessment should consider the maturity of the project, including release history, roadmap availability, versioning practices and evidence of operational use. Indicators such as the number of active contributors, governance transparency, institutional backing and the “bus factor” provide insight into long-term sustainability and associated risks.

- Security and risk management. The project should have documented security practices, including vulnerability reporting mechanisms, patching policies and incident response procedures. Known risks, external dependencies and lifecycle considerations (including end-of-life or migration paths) should be evaluated early in the selection process.
- Technical quality and operational fitness. Software quality should be assessed based on documentation availability, test coverage, automation (e.g. CI/CD), performance at NMHS-relevant data volumes, and ease of deployment, operation and maintenance. Evidence of production use within the WMO community or by peer organizations is a strong positive indicator.
- Organizational readiness and total cost of ownership (TCO). Adopting organizations should evaluate internal capacity, including staff skills, support models, infrastructure requirements and TCO. This includes training needs, long-term maintenance effort and the ability to contribute fixes or enhancements upstream where appropriate.

To support early-stage identification and comparison, Members and WMO activities may also consult community-maintained software catalogues, such as the FOSS4G Observatory (https://project.oss4geo.org/). These platforms provide structured metadata for a wide range of open-source software projects, including functional scope, licensing, governance characteristics, development activity and dependency relationships. Such catalogues can assist with preliminary screening and ecosystem-level understanding, but the information they provide should be treated as indicative and complementary.

For consistency and transparency, WMO encourages the use of a common evaluation checklist or rubric, such as the one provided in Annex A, and promotes rolling reviews of selected software to account for evolving requirements, standards and ecosystem changes. This assessment process does not seek to formally “approve” specific software for inclusion in WMO Technical Regulations, but rather to provide confidence-based guidance and shared criteria to support informed decision-making and harmonized adoption across the WMO ecosystem.

Reference Implementations

Reference Implementations (RIs) are key resources for the precise implementation of a given standard or requirement set. Benefits of RIs include (but are not limited to):

- validating the functionality and capability of a given standard (for example, for data encodings or interfaces)
- providing a publicly available FOSS resource that is definitive and can be studied to better understand and/or implement a standard
- providing an implementation that may be used by Members as desired

RIs developed alongside a given specification are especially useful for agile, rapid and iterative feedback during the specification definition phase, leading to the approval of Technical Regulations with confidence. It is therefore strongly recommended to develop RIs as part of WMO Technical Regulation development to ensure fit for purpose and functionality. Citing a RI as part of a WMO Technical Regulation should not be required, but allowed if applicable.

NOTE	They key purpose of an RI is functionality coverage. They need not be suitable for

> mission critical or production environments.

Compliance (data exchange)

- **Standards alignment**: The software adheres to WMO technical standards and common industry specifications, unifying data exchange interfaces and formats.
- **Data protection**: It complies with international conventions and national data protection regulations by implementing data desensitization, encrypted data transmission, and secure storage of sensitive meteorological data.
- **Cross-border data security**: It meets data cross-border transfer requirements in operational regions by establishing a full lifecycle data-security management mechanism. This mechanism covers data collection, processing, sharing, and archiving in line with WMO data policies.

Software evaluation

- **Core Evaluation Checklist**:
  - *Technical Quality*: Functionality completeness and accuracy for meteorological use cases, performance when processing massive meteorological datasets (e.g., real-time observational data, forecast model outputs), reliability under peak load conditions, security, maintainability, and compatibility with existing meteorological systems.
  - *Compliance*: License is OSI-approved with no license conflicts, and complies with WMO data policies and relevant regulations.
  - *Community Sustainability*: Recent community activity (within the last 6–12 months), stability of the maintenance team, regularity of version releases, and maturity of the ecosystem.
  - *Cost-effectiveness*: Controllable costs across the software lifecycle (deployment, operation, maintenance, training, migration, and decommissioning).
  - *Business Fit*: Aligns with existing meteorological operational processes, good usability for forecasters and data analysts, and support for future expansion needs.
- **Confidence Assurance:**
  - Adopt standardized evaluation processes and quantifiable metrics to ensure objectivity and repeatability of results.
  - Validate core functionality through a Proof-of-Concept (PoC) approach, incorporating feedback collected from real-world operational scenarios.
  - Conduct multi-dimensional verification, including code audits, security vulnerability scans, and in-depth open source community research, to confirm the reliability of the software.

Readiness

Completed drafting of requirements specification documents, and form a professional evaluation team (including technical, business, legal, and operations personnel).

Set up a simulated meteorological operations testing environment, and prepare test data (including extreme weather scenario data) and necessary evaluation tools.

Define clear priorities for software requirements and establish final decision criteria, such as

approval for full adoption, conditional adoption, or rejection. Document the rationale for all decisions for audit purposes.

Bus/retirement factor

Business Fit and Retirement Considerations

- Business Fit
  - Offers seamless integration with existing meteorological operational processes, minimizing the need for process adjustments.
  - Supports custom development and feature expansion to meet evolving needs (such as business scaling and new scenario integration).
  - Aligns with users' habits in meteorological operations by providing a user-friendly interface and flexible configuration options.
- Retirement Planning
  - Establish a clear data archiving plan to ensure secure storage and traceability of historical meteorological data after the software is decommissioned.
  - Proactively assess the costs of migrating to new systems, and establish a smooth migration path to ensure business continuity.
  - Define clear responsibilities and standardized procedures for software retirement to mitigate potential risks (e.g., data leakage, system conflicts, and service disruptions).

Case study: WIS2, FOSS, and agile development

The WMO Information System 2.0 (WIS 2.0) is the framework for WMO data sharing in the 21st century for all WMO domains and disciplines. It supports the WMO Unified Data policy and the Global Basic Observing Network (GBON) and makes international, regional, and national data sharing simple, effective, and inexpensive. The idea that no Member should be left behind and the objective of lowering the barrier to adoption has been at the core of WIS 2.0 development. These objectives inspire the principles underpinning the WIS 2.0 technical framework, such as adopting open standards and Web technologies to facilitate sharing of increasing variety and volume of real-time data.[2]

The WMO Executive Council, through Resolution 34 (EC-76) - Implementation plan update of the WMO Information System 2.0, endorsed the WMO Information System 2.0 (WIS2) implementation plan. The Resolution also recognized the importance of establishing a pilot phase to develop the WIS2 infrastructure and begin testing it, in order to be ready for a pre-operational phase in 2024, and then for the transition starting in 2025.

The pilot phase was completed at the end of 2023, with several Members collaborating in building the WIS2 infrastructure. Each Member had a different role in the WIS2 framework and implemented a specific component. Starting in January 2024, the implementation of WIS2 entered the pre-operational phase in preparation for transitioning to WIS2. On 01 January 2025, WIS2 became operational.[3]

WIS2 is comprised of a number of key standards, including (but are not limited to):

- Internet and messaging protocols for data and metadata transmission (HTTP, MQTT)
- metadata encodings for dataset descriptions (WMO Core Metadata Profile) and notifications (WIS2 Notification Message)
- topic structures for real-time notifications (WIS2 Topic Hierarchy)
- interfaces and APIs for Global Services (Global Discovery Catalogue)

Throughout the WIS2 Implementation timeline, the architecture, standards definitions as well as key software components were developed, deployed and tested in parallel, in an agile manner. These software include (but are not limited to):

Software	WIS2 component(s)
wis2box	WIS2 Node
wis2-gdc	Global Discovery Catalogue
csv2bufr	CSV to BUFR encoding tool for station data
wis2downloader	Python package for downloading real-time data from WIS2
pywis-pubsub	Python package providing notification message publishing, subscription and data download capability in WIS2

Driven by collaboration, the development and testing of FOSS significantly helped with early assessment of the WIS2 standards in development. As a result, WIS2 standards were being tested in an agile environment and rapid feedback was provided that helped refine, adjust and improve the standards early and often.

WIS2 FOSS implementations have resulted in a more stable and robust program delivery with significantly reduced risk to Members. In addition, some tools have emerged as Reference Implementations. WIS2 Technical Regulations were approved with confidence as a result of the agile development of Open Standards and Open Source.

[1] https://www.osgeo.org/resources/project-graduation-checklist/

[2] https://wmo.int/wis-20

[3] https://wmo-im.github.io/wis2-transition-guide/transition-guide/wis2-transition-guide-APPROVED.html#_1_introduction

References

- Fogel, Karl: Producing Open Source Software: How to Run a Successful Free Software Project (2024) [1]

[1] https://producingoss.com

Annex A: FOSS evaluation rubric

Category	Indicator
License Compatibility	License is permissive (MIT, BSD, Apache 2.0)
	Compatible with NMHS mandates and data policies
	No restrictions on data sharing or commercial use
	License is OSI-approved
	License is clearly stated (no license = reject)
Security	Clear security vulnerability policy/process
	Prompt patch releases (within days)
	No known unpatched CVEs
	Regular security checks (e.g., daily or automated)
Project Health	Public issue tracker
	Source code under version control (e.g. on GitHub, GitLab, Codeberg or other version control system)
	Active communication channels (forum, mailing list, chat)
	Wiki or documentation site
	Contribution guidelines or agreements
Community Health	Multiple active contributors
	Supported by major NMHSs, WMO Programmes, or recognized consortia
	Low bus factor (not reliant on one person)
	Code of conduct available
Openness & Governance	Transparent governance model
	Open contribution process
	Alignment with WMO standards (BUFR, GRIB2, NetCDF)
	Decision-making process documented
	Source code openly available (mandatory)
Technical Quality & Fit	High test coverage
	Continuous Integration (CI) in place
	Comprehensive documentation (user, developer, admin)
	Proven ability to handle NMHS-scale data volumes
	Validated in operational NMHS environments
	Clear release lifecycle and package management

Annex B: Examples of FOSS managed / offered by Member organizations

Organization	Platform	URL	Projects(s)
Environment and Climate Change Canada, Meteorological Service of Canada	GitHub	https://github.com/ECCC-MSC	• https://github.com/ECCC-MSC/msc-pygeoapi • https://github.com/ECCC-MSC/libecbufr • https://github.com/ECCC-MSC/msc-wis2node

Organization	Platform	URL	Projects(s)
NSF Unidata	GitHub	https://github.com/Unidata	• TODO1 • TODO2